

# Selection Sort

Scan the entire unsorted suffix, remember its minimum index, and swap that minimum into the next final position.

## How to use this chapter:

Learn the idea in plain language, read each diagram, predict every trace row, then study the commented Java and attempt the exam-style questions.

## The whole process at a glance

Scan the entire unsorted suffix, remember its minimum index, and swap that minimum into the next final position.

Find the smallest remaining card before you move anything. Put it in the next empty final position. Repeat with the cards still waiting.

Read down the page: scan the entire suffix after the green sorted prefix, select its blue minimum, then exchange it with the orange destination. A no-swap pass still performs its full scan.

Green: sorted / found

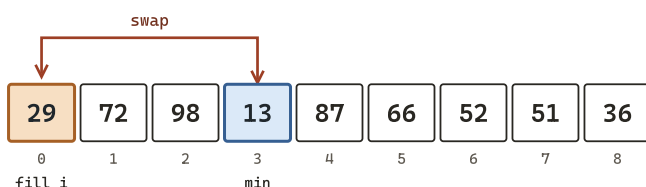
Blue: chosen value / midpoint

Orange: destination / held key

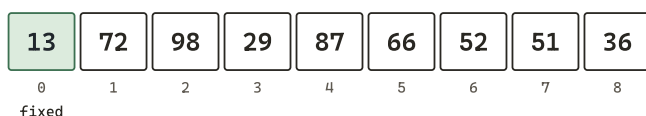
Crossed: ruled out

· = conceptual gap

### Pass 1: fill index 0

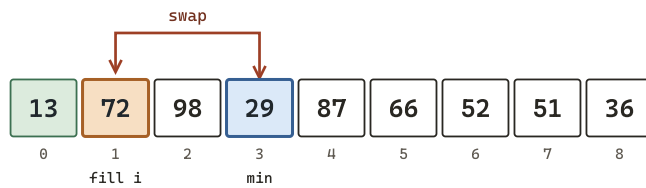


Minimum 13 at index 3; swap 29 ↔ 13. Compare all 8 remaining candidates before moving values.

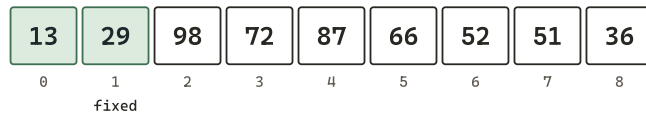


Position 0 is final. The displaced value returns to the minimum's old position.

## Pass 2: fill index 1

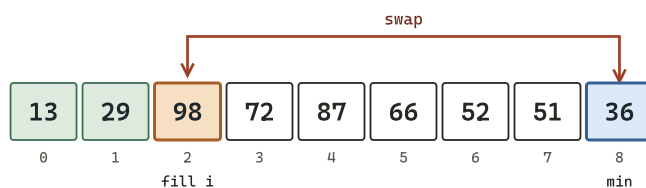


Minimum 29 at index 3; swap 72  $\leftrightarrow$  29. Compare all 7 remaining candidates before moving values.

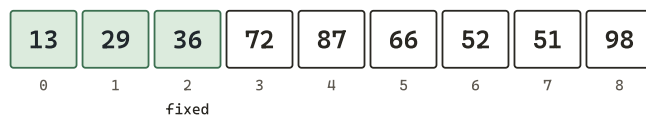


Position 1 is final. The displaced value returns to the minimum's old position.

## Pass 3: fill index 2

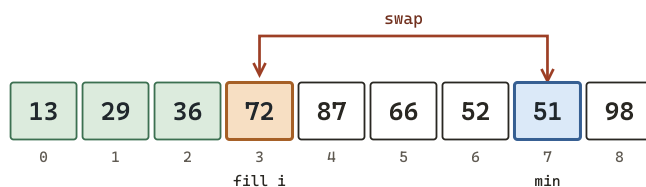


Minimum 36 at index 8; swap 98  $\leftrightarrow$  36. Compare all 6 remaining candidates before moving values.

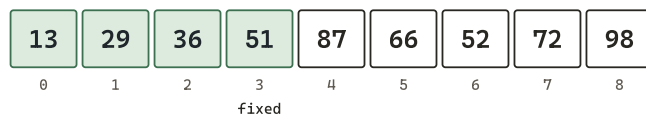


Position 2 is final. The displaced value returns to the minimum's old position.

## Pass 4: fill index 3

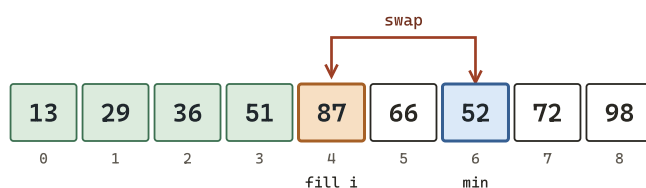


Minimum 51 at index 7; swap 72  $\leftrightarrow$  51. Compare all 5 remaining candidates before moving values.

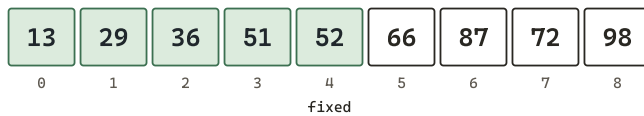


Position 3 is final. The displaced value returns to the minimum's old position.

## Pass 5: fill index 4

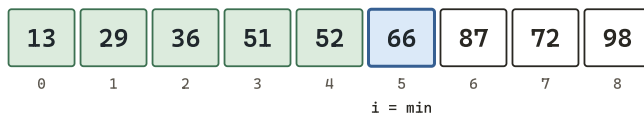


Minimum 52 at index 6; swap 87  $\leftrightarrow$  52. Compare all 4 remaining candidates before moving values.

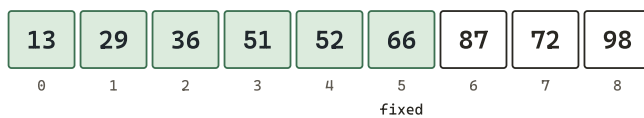


Position 4 is final. The displaced value returns to the minimum's old position.

### Pass 6: fill index 5

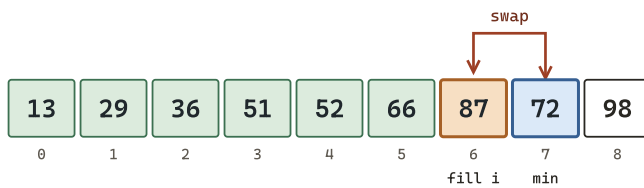


Minimum 66 is already at index 5: no swap. Compare all 3 remaining candidates before moving values.

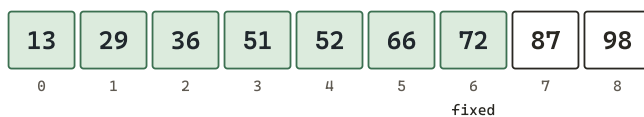


Position 5 is final. The array stays unchanged.

### Pass 7: fill index 6

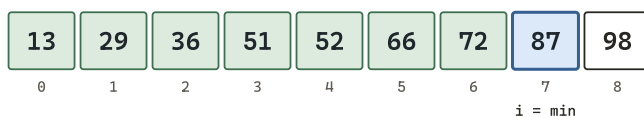


Minimum 72 at index 7; swap 87 ↔ 72. Compare all 2 remaining candidates before moving values.

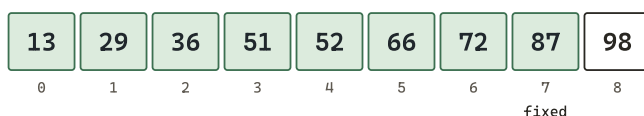


Position 6 is final. The displaced value returns to the minimum's old position.

### Pass 8: fill index 7



Minimum 87 is already at index 7: no swap. Compare all 1 remaining candidates before moving values.



Position 7 is final. The array stays unchanged.

Sorted: [13, 29, 36, 51, 52, 66, 72, 87, 98]. 36 key comparisons; 6 actual swaps. The last value is already in place after pass 8.

Sorting / Selection sort · The textbook definition

# The textbook definition

**Selection sort** is a comparison-based sorting algorithm that divides the array into a sorted prefix on the left and an unsorted suffix on the right. In each iteration it *scans the whole unsorted suffix* to find the smallest value, then swaps that value into the first position of the suffix. The sorted prefix therefore grows by one cell per iteration, and every cell in it is already in its final place.

|                                                                                                                                       |                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>Sorted prefix</b><br>The cells $0 \dots i - 1$ . They hold the smallest values, in order, and never change again.                  | <b>Unsorted suffix</b><br>The cells $i \dots n - 1$ . The next minimum is somewhere in here.                            |
| <b>Boundary <math>i</math></b><br>The first cell of the unsorted suffix, which is the cell about to be filled.                        | <b>Scan</b><br>Looking at every cell of the suffix once, comparing it with the best candidate so far.                   |
| <b><code>minIndex</code></b><br>The <i>position</i> of the smallest value found so far. We remember where it is, not just what it is. | <b>Swap</b><br>Exchanging <code>array[i]</code> with <code>array[minIndex]</code> . There is at most one per iteration. |

The name says what it does: each iteration **selects** the right value for the next position, rather than shuffling values into place as bubble and insertion sort do.

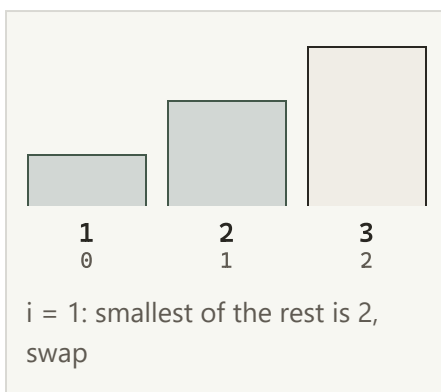
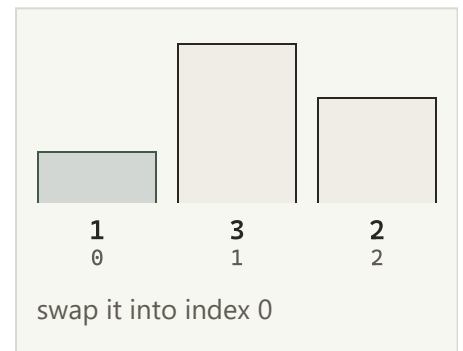
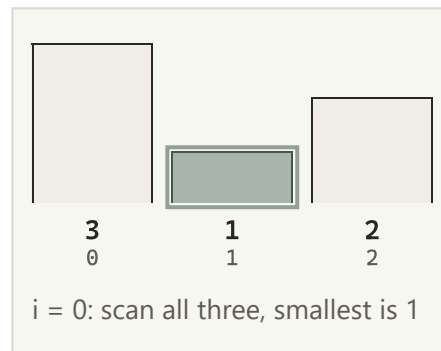
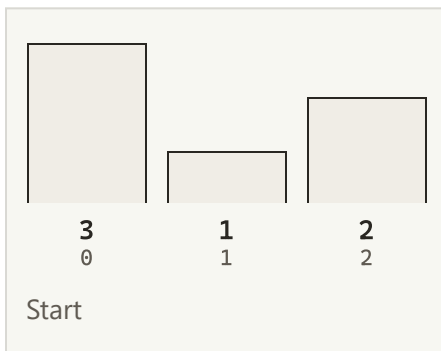
Sorting / Selection sort · In plain language

# In plain language

Keep choosing the smallest card from the pile that is left, and lay it down next in the row.

Imagine a pile of numbered cards on a table. Look through **all** of them and pull out the smallest; put it at the left end of your row. Now look through the cards that are left, pull out the smallest of those, and put it next in the row. Keep going until the pile is empty. Your row is sorted.

In an array there is no separate table: the “pile” is the part of the array to the right of the boundary, and “putting it in the row” means swapping the chosen value into the boundary cell. Whatever was sitting in that cell goes to the place the chosen value came from.



■ smallest found so far    ■ final position

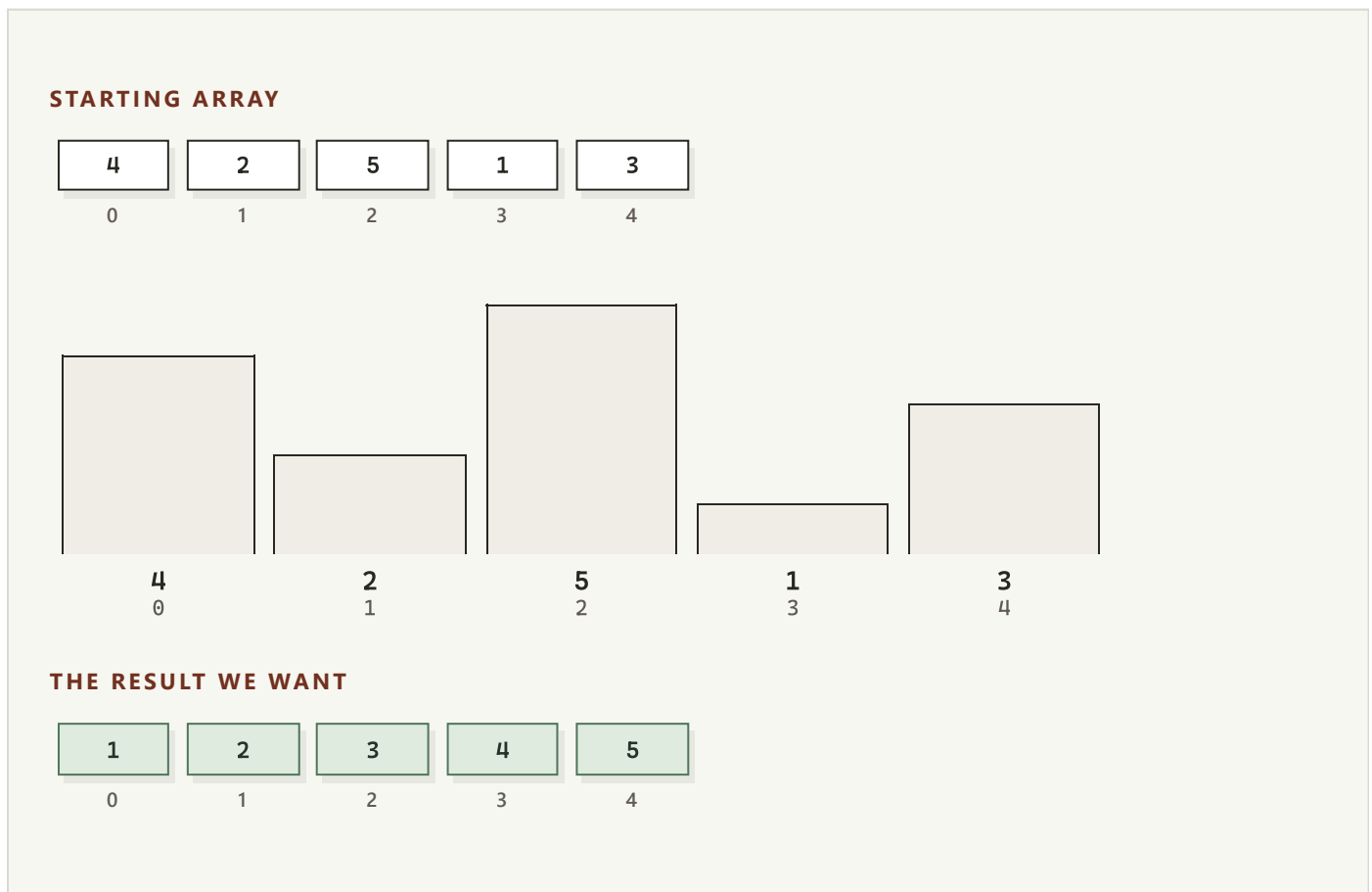
Two iterations were enough for three values: the last cell is automatically correct when all the others are, so the loop only runs  $n - 1$  times.

**Detailed reference example:** this part introduces its own input, shown below. The opening whole-run map and guided walkthrough keep their original input throughout; each later worked example states its new input explicitly.

*Sorting / Selection sort · The array we will sort*

# The array we will sort

The same five values again:  $[4, 2, 5, 1, 3]$ , sorted from smallest to largest.



It is the same array that bubble sort and insertion sort use, so you can compare the three methods on identical data. One value deserves attention: the 1 is the smallest and sits at index 3. Bubble sort moves it one place per pass. Selection sort will find it and move it to the front **in a single swap**.

## Sorting / Selection sort · The main idea

### The main idea

Selection sort splits the array in two with a movable boundary. Everything left of the boundary is **finished**; everything right of it is untouched. The whole algorithm is: *find the smallest value on the right, put it at the boundary, move the boundary one cell right*.

#### One scan for the minimum

Here is the scan for  $i = 0$ . We start by assuming index 0 holds the minimum, then look at every other cell. Whenever we meet something smaller we move our “minimum so far” marker to it. Only when the scan ends do we know the true minimum and swap once.



## The prefix is final, not just sorted

This is the biggest difference from insertion sort. In insertion sort the left part is sorted *among itself* but later values may still be inserted in the middle. In selection sort the left part holds the globally smallest values, so a finished cell is **never touched again**.

After  $i = 0$  · swapped index 0 with index 3



After  $i = 1$  · the minimum was already in place, so nothing moved



After  $i = 2$  · swapped index 2 with index 4



After  $i = 3$  · the minimum was already in place, so nothing moved



## The cost of choosing

Finding the minimum means looking at every cell of the suffix, every time, even if the array was sorted to begin with. The algorithm has no way to know the minimum is already in place until it has checked. That is what you pay for the very small number of swaps.

**The repeating recipe:** set `minIndex = i` → scan `i + 1 .. n - 1`, updating `minIndex` whenever a smaller value appears → swap `array[i]` with `array[minIndex]` → move `i` right.

*Sorting / Selection sort · Scan, select, swap*

## Scan, select, swap

Selection sort separates *finding* from *moving*. Seeing them as two phases makes the code and the trace easy to follow.

### Comparison

A question inside the scan: `array[j] < array[minIndex]`. Every iteration makes  $n - 1 - i$  of them, regardless of the data.

### Selection

Deciding which cell holds the minimum. Nothing in the array moves during this phase.

### Swap

The only movement: one exchange per iteration at most, and none if the minimum is already at `i`.

### Iteration

One value of `i`: a full scan plus (maybe) one swap.

## Why remember the index, not the value?

Knowing that the smallest value is 1 is not enough: to swap we must know *where* the 1 is. That is why the code keeps `minIndex`. A common slip is to store `min = array[j]` and then be unable to find its position.

## One swap can move a value a long way

Bubble sort moves values one neighbour at a time. Selection sort's swap jumps: in iteration 0 the 4 and the 1 trade places across three cells. That long jump is what keeps the swap count tiny, but it also has a price, which you will meet in the section on stability: jumping over equal values can reorder them.



| STEP | CODE                                                                               | EFFECT ON [4, 2, 5, 1, 3]                                |
|------|------------------------------------------------------------------------------------|----------------------------------------------------------|
| scan | <pre>for (j = i + 1; j &lt; n; j++) if (a[j] &lt; a[minIndex]) minIndex = j;</pre> | minIndex ends at 3 (the value 1). Nothing has moved yet. |
| swap | <pre>temp = a[i]; a[i] = a[minIndex]; a[minIndex] = temp;</pre>                    | [1, 2, 5, 4, 3]: the 4 and the 1 trade places.           |

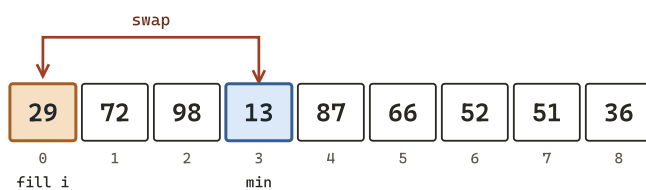
Counting rule: **comparisons** for this array total 10 no matter what; **swaps** total 2. Compare that with bubble sort's 6 swaps on the same array.

## Guided walkthrough: predict the next major step

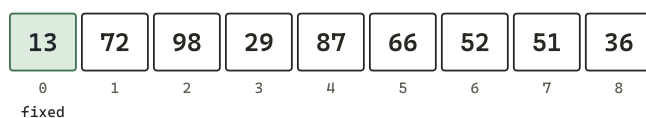
Predict the next completed pass, then advance. Each scene keeps all its movements together.

### Guided walkthrough: major steps

#### Pass 1: fill index 0



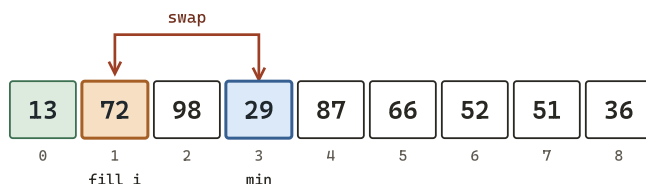
Minimum 13 at index 3; swap 29 ↔ 13. Compare all 8 remaining candidates before moving values.



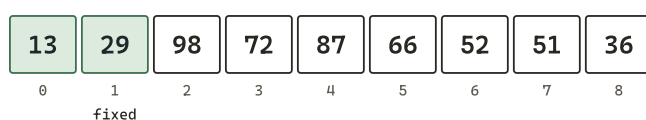
Position 0 is final. The displaced value returns to the minimum's old position.

Minimum 13 at index 3; swap 29 ↔ 13.

#### Pass 2: fill index 1



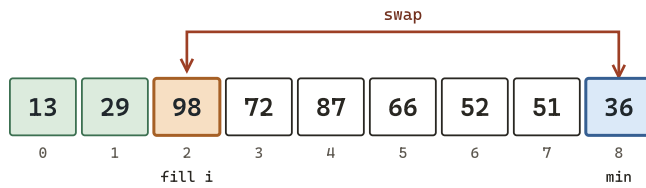
Minimum 29 at index 3; swap 72 ↔ 29. Compare all 7 remaining candidates before moving values.



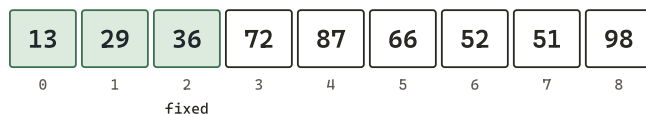
Position 1 is final. The displaced value returns to the minimum's old position.

Minimum 29 at index 3; swap 72 ↔ 29.

### Pass 3: fill index 2



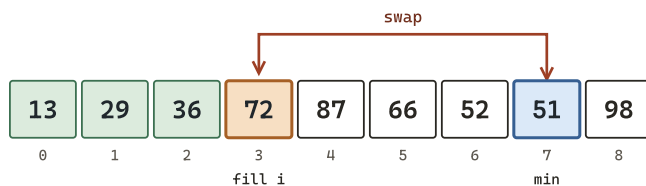
Minimum 36 at index 8; swap 98 ↔ 36. Compare all 6 remaining candidates before moving values.



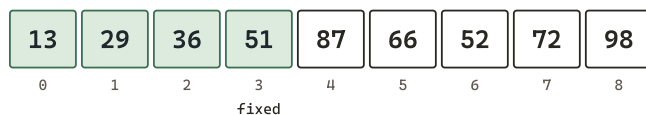
Position 2 is final. The displaced value returns to the minimum's old position.

Minimum 36 at index 8; swap 98 ↔ 36.

### Pass 4: fill index 3



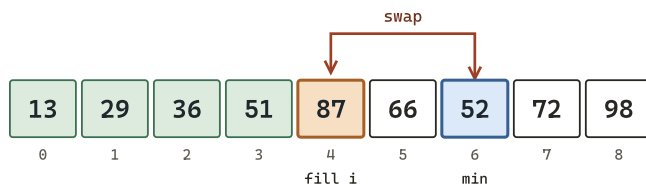
Minimum 51 at index 7; swap 72 ↔ 51. Compare all 5 remaining candidates before moving values.



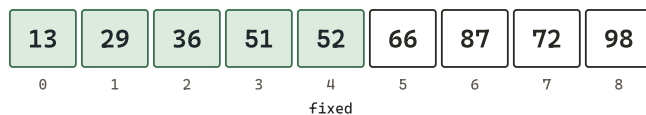
Position 3 is final. The displaced value returns to the minimum's old position.

Minimum 51 at index 7; swap 72 ↔ 51.

### Pass 5: fill index 4



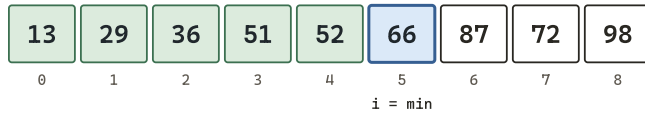
Minimum 52 at index 6; swap 87 ↔ 52. Compare all 4 remaining candidates before moving values.



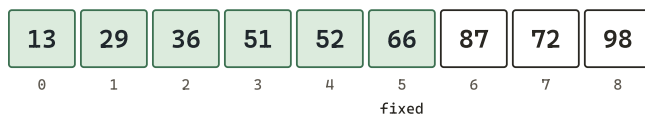
Position 4 is final. The displaced value returns to the minimum's old position.

Minimum 52 at index 6; swap 87 ↔ 52.

### Pass 6: fill index 5



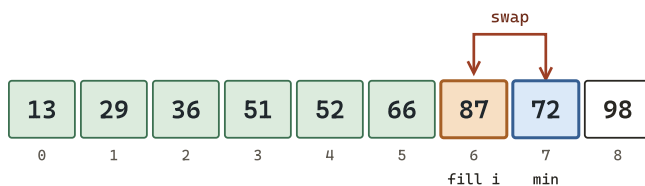
Minimum 66 is already at index 5: no swap. Compare all 3 remaining candidates before moving values.



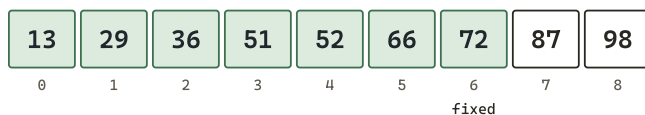
Position 5 is final. The array stays unchanged.

Minimum 66 is already at index 5: no swap.

### Pass 7: fill index 6



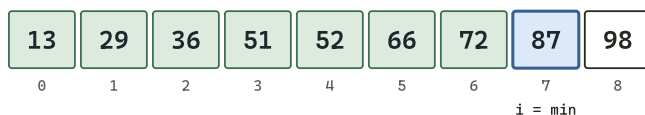
Minimum 72 at index 7; swap 87 ↔ 72. Compare all 2 remaining candidates before moving values.



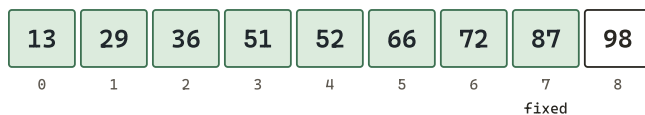
Position 6 is final. The displaced value returns to the minimum's old position.

Minimum 72 at index 7; swap 87 ↔ 72.

### Pass 8: fill index 7



Minimum 87 is already at index 7: no swap. Compare all 1 remaining candidates before moving values.



Position 7 is final. The array stays unchanged.

Minimum 87 is already at index 7: no swap.

▼ Full trace table - select a row to jump to its scene

|      |         |         |                                      |             |
|------|---------|---------|--------------------------------------|-------------|
|      |         |         |                                      |             |
| 1    | 13 at 3 | 0 ↔ 3   | [13, 72, 98, 29, 87, 66, 52, 51, 36] | 8           |
| 2    | 29 at 3 | 1 ↔ 3   | [13, 29, 98, 72, 87, 66, 52, 51, 36] | 7           |
| 3    | 36 at 8 | 2 ↔ 8   | [13, 29, 36, 72, 87, 66, 52, 51, 98] | 6           |
| 4    | 51 at 7 | 3 ↔ 7   | [13, 29, 36, 51, 87, 66, 52, 72, 98] | 5           |
| 5    | 52 at 6 | 4 ↔ 6   | [13, 29, 36, 51, 52, 66, 87, 72, 98] | 4           |
| 6    | 66 at 5 | No swap | [13, 29, 36, 51, 52, 66, 87, 72, 98] | 3           |
| 7    | 72 at 7 | 6 ↔ 7   | [13, 29, 36, 51, 52, 66, 72, 87, 98] | 2           |
|      | 87 at 7 | No swap | [13, 29, 36, 51, 52, 66, 72, 87, 98] | 1           |
| PASS | MINIMUM | ACTION  | ARRAY AFTER PASS                     | COMPARISONS |

## The trace at a glance

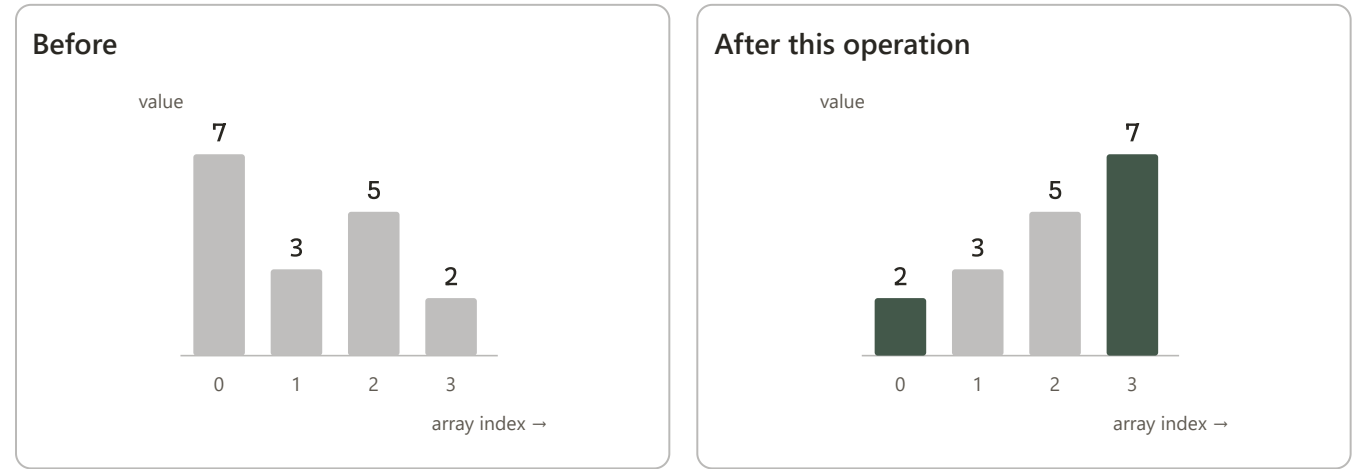
| PASS | MINIMUM | ACTION | ARRAY AFTER PASS                     | COMPARISONS |
|------|---------|--------|--------------------------------------|-------------|
| 1    | 13 at 3 | 0 ↔ 3  | [13, 72, 98, 29, 87, 66, 52, 51, 36] | 8           |

| PASS | MINIMUM | ACTION  | ARRAY AFTER PASS                     | COMPARISONS |
|------|---------|---------|--------------------------------------|-------------|
| 2    | 29 at 3 | 1 ↔ 3   | [13, 29, 98, 72, 87, 66, 52, 51, 36] | 7           |
| 3    | 36 at 8 | 2 ↔ 8   | [13, 29, 36, 72, 87, 66, 52, 51, 98] | 6           |
| 4    | 51 at 7 | 3 ↔ 7   | [13, 29, 36, 51, 87, 66, 52, 72, 98] | 5           |
| 5    | 52 at 6 | 4 ↔ 6   | [13, 29, 36, 51, 52, 66, 87, 72, 98] | 4           |
| 6    | 66 at 5 | No swap | [13, 29, 36, 51, 52, 66, 87, 72, 98] | 3           |
| 7    | 72 at 7 | 6 ↔ 7   | [13, 29, 36, 51, 52, 66, 72, 87, 98] | 2           |
| 8    | 87 at 7 | No swap | [13, 29, 36, 51, 52, 66, 72, 87, 98] | 1           |

## Read the algorithm as a bar graph

Bar height means **value**; horizontal position means **index**. All panels below use the same height scale. A taller bar is not “more sorted”.

Find the shortest bar, then exchange two positions



Scan all remaining bars: 2 is the minimum at index 3. After the scan, swap 7 and 2 once. The displaced 7 goes back to index 3. The scan itself changes only minIndex; the bars do not move while you search.

**Check yourself:** identify which indexes are guaranteed final, which values were merely inspected, and whether the operation swapped, shifted or copied data. A sorted ascending array has non-decreasing heights, including equal-height neighbours.

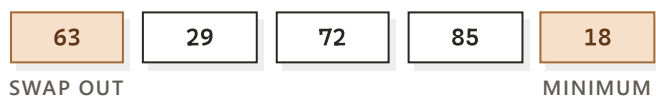
Sorting / Selection sort

# Select the smallest remaining value

Selection sort scans the unsorted section for its smallest value, then swaps that value into the next open position. The sorted section grows from left to right.

## Illustration · choose the first minimum

In [63, 29, 72, 85, 18], begin with 63 as the assumed minimum. Comparing the remaining values eventually finds 18 at index 4.



Swap 63 and 18:



## Trace table

| POSITION | MINIMUM FOUND | ACTION         | ARRAY AFTER PLACEMENT | COMPARISONS |
|----------|---------------|----------------|-----------------------|-------------|
| 0        | 18 at index 4 | Swap 63 ↔ 18   | [18, 29, 72, 85, 63]  | 4           |
| 1        | 29 at index 1 | No swap needed | [18, 29, 72, 85, 63]  | 3           |
| 2        | 63 at index 4 | Swap 72 ↔ 63   | [18, 29, 63, 85, 72]  | 2           |
| 3        | 72 at index 4 | Swap 85 ↔ 72   | [18, 29, 63, 72, 85]  | 1           |

# Implementation

## Selection sort

```
public static void selectionSort(int[] arr) {
    // i is the next position to fill in the sorted prefix.
    for (int i = 0; i < arr.length - 1; i++) {
        int minIndex = i;
        // Search the unsorted suffix for its smallest value.
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[j] < arr[minIndex]) minIndex = j;
        }
        // Place the selected minimum at i; avoid a self-swap.
        if (minIndex != i) {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }
}
```

## ▼ Recursive selection sort

First call: `selectionSortRec(arr, 0)`.

## Java

```
public static void selectionSortRec(int[] arr, int start) {
    // Base case: fewer than two values remain.
    if (start ≥ arr.length - 1) return;

    int minIndex = start;
    // Find the minimum in start..last.
    for (int j = start + 1; j < arr.length; j++) {
        if (arr[j] < arr[minIndex]) minIndex = j;
    }
    if (minIndex != start) {
        int temp = arr[start];
        arr[start] = arr[minIndex];
        arr[minIndex] = temp;
    }
    // The prefix gained one fixed value; solve the suffix.
    selectionSortRec(arr, start + 1);
}
```

## Write it from scratch

- 1 Choose the next position  $i$ .
- 2 Assume  $i$  holds the minimum.
- 3 Scan the suffix and update `minIndex`.
- 4 Swap the minimum into  $i$ .

### Ascending ↔ descending

For ascending, select the minimum with  $<$ . For descending, select the maximum with  $>$ .

**Same array, opposite choice:** on  $[63, 29, 72, 85, 18]$  with  $i = 0$ , ascending selects 18 (index 4) and descending selects 85 (index 3).

## Debug checklist

- Store an index, not only the minimum value.
- Start the inner loop at  $i + 1$ .
- Swap only after the scan finishes.

## Worked example · full run on $[4, 2, 5, 1, 3]$

The orange divider marks the end of the sorted prefix. Dashed-outline cells mark the position  $i$  being filled; the raised cell is the minimum found.

$i = 0$  · fill index 0 with the smallest of the remaining values



Scan indexes 1–4 (4 comparisons). The smallest remaining value is 1 at index 3. Swap index 0 (4) with index 3 (1).



$i = 1$  · fill index 1 with the smallest of the remaining values

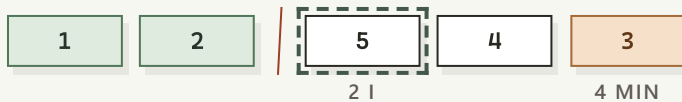


Scan indexes 2–4 (3 comparisons). The smallest remaining value is 2 at index 1. Index 1 already holds the minimum, so no swap is needed.





**i = 2 · fill index 2 with the smallest of the remaining values**



Scan indexes 3–4 (2 comparisons). The smallest remaining value is 3 at index 4. Swap index 2 (5) with index 4 (3).



**i = 3 · fill index 3 with the smallest of the remaining values**



Scan index 4 (1 comparison). The smallest remaining value is 4 at index 3. Index 3 already holds the minimum, so no swap is needed.



| I            | MINIMUM FOUND | ACTION         | ARRAY AFTER     | COMPARISONS |
|--------------|---------------|----------------|-----------------|-------------|
| 0            | 1 at index 3  | Swap           | [1, 2, 5, 4, 3] | 4           |
| 1            | 2 at index 1  | No swap        | [1, 2, 5, 4, 3] | 3           |
| 2            | 3 at index 4  | Swap           | [1, 2, 3, 4, 5] | 2           |
| 3            | 4 at index 3  | No swap        | [1, 2, 3, 4, 5] | 1           |
| <b>Total</b> | —             | <b>2 swaps</b> | Sorted          | <b>10</b>   |

**Sorted: 10 comparisons but only 2 swaps, compared with 6 swaps for bubble sort on the same array.**

## Why selection sort is not stable

The swap moves a value across everything between the two positions, which can jump it over an equal value. Label two equal 2s as  $2^a$  and  $2^b$  and sort  $[2^a, 2^b, 1]$ : the minimum 1 is swapped with  $2^a$ .

**Before**

**After the swap at i = 0**



2<sup>b</sup> now sits before 2<sup>a</sup>, so the equal values changed relative order. Insertion sort avoids this because it never moves a value past an equal one. Merge sort avoids it because, on a tie, the merge takes the left value first ( $\leq$ ).

Best case · already sorted

For [1, 2, 3, 4], each scan confirms that the first unsorted value is already the minimum.

The sorted boundary grows, but the scans do not disappear

1

2

3

4

I = 0 · 3 CHECKS · 2 CHECKS · 1 CHECK · FINAL

Comparisons: 3 + 2 + 1 = 6 · swaps: 0 · still  $O(n^2)$ .

| I | SUFFIX SCANNED | MININDEX | COMPARISONS | SWAP? |
|---|----------------|----------|-------------|-------|
| 0 | [1, 2, 3, 4]   | 0        | 3           | No    |
| 1 | [2, 3, 4]      | 1        | 2           | No    |
| 2 | [3, 4]         | 2        | 1           | No    |

|                                             |                                             |
|---------------------------------------------|---------------------------------------------|
| BEST<br><b><math>O(n^2)</math></b>          | AVERAGE<br><b><math>O(n^2)</math></b>       |
| WORST<br><b><math>O(n^2)</math></b>         | STABLE<br><b>No</b>                         |
| IN-PLACE<br><b>Yes</b>                      | ADAPTIVE<br><b>No</b>                       |
| ITERATIVE SPACE<br><b><math>O(1)</math></b> | RECURSIVE SPACE<br><b><math>O(n)</math></b> |

## Questions

### ▼ Why might selection sort be chosen over bubble sort?

Selection sort makes at most one swap per outer iteration, so it can be useful when writes or swaps are expensive.

### ▼ Why is it not adaptive?

It scans the entire remaining unsorted section to find the minimum even when the array is already sorted.

### Selection sort in one sentence

Find the smallest value in the unsorted suffix and swap it into the next position of the sorted prefix.

## Fully annotated Java: connect each line to the movement

● ● ● java

```
public static void selectionSort(int[] arr) {
    // Fill each final position from left to right.
    for (int i = 0; i < arr.length - 1; i++) {
        int min = i; // Store an INDEX, not a value.
        // Scan the complete unsorted suffix before swapping anything.
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[j] < arr[min]) min = j;
        }
        // At most one swap per pass; skip a redundant self-swap.
        if (min != i) {
            int temp = arr[i]; arr[i] = arr[min]; arr[min] = temp;
        }
    }
}
```

## Write it from scratch

- Choose the next final position.
- Initialize the minimum index to it.
- Scan the complete suffix.

- Remember better candidate indexes.
- Perform at most one final swap.

## Recursion, calls and returns

java

```
public static void selectionSortRec(int[] arr, int i) {
    // Zero or one position remains to fill.
    if (i ≥ arr.length - 1) return;
    int min = i;
    for (int j = i + 1; j < arr.length; j++) {
        if (arr[j] < arr[min]) min = j;
    }
    if (min ≠ i) {
        int temp = arr[i]; arr[i] = arr[min]; arr[min] = temp;
    }
    // The completed prefix grows; the remaining suffix shrinks.
    selectionSortRec(arr, i + 1);
    // First call: selectionSortRec(arr, 0).
}
```

### CALL STACK - NEWEST FRAME AT THE TOP

**selectionSortRec(i=1)**

Scan suffix 1..4 and fill index 1

**selectionSortRec(i=0)**

Waiting: Child fills remaining positions 1..4

Unlike iterative auxiliary space, suspended recursive frames require  $O(n)$  stack space in the worst case.

## A complete runnable Java example

Save as `SelectionDemo.java`. The code includes the method(s) and a main entry point.

```
import java.util.Arrays;

public class SelectionDemo {
    public static void selectionSort(int[] arr) {
        // Fill each final position from left to right.
        for (int i = 0; i < arr.length - 1; i++) {
            int min = i; // Store an INDEX, not a value.
            // Scan the complete unsorted suffix before swapping anything.
            for (int j = i + 1; j < arr.length; j++) {
                if (arr[j] < arr[min]) min = j;
            }
            // At most one swap per pass; skip a redundant self-swap.
            if (min != i) {
                int temp = arr[i]; arr[i] = arr[min]; arr[min] = temp;
            }
        }
    }

    public static void main(String[] args) {
        int[] values = {4, 2, 5, 1, 3};
        selectionSort(values);
        System.out.println(Arrays.toString(values));
    }
}
```

```
[1, 2, 3, 4, 5]
```

## Sorting / Selection sort · Properties

# Properties

|                                          |                                            |
|------------------------------------------|--------------------------------------------|
| BEST TIME<br><b><math>O(n^2)</math></b>  | AVERAGE TIME<br><b><math>O(n^2)</math></b> |
| WORST TIME<br><b><math>O(n^2)</math></b> | EXTRA SPACE<br><b><math>O(1)</math></b>    |
| STABLE<br><b>No</b>                      | IN-PLACE<br><b>Yes</b>                     |
| ADAPTIVE<br><b>No</b>                    | SWAPS<br><b><math>\leq n - 1</math></b>    |

## Not stable

The long-distance swap can jump a value over an equal one. In  $[2^a, 2^b, 1]$  the first swap moves  $2^a$  behind  $2^b$ , and nothing later repairs it. A stable version exists (shift instead of swap), but that is a different algorithm.

## In-place

Only the array and a few variables are used:  $O(1)$  extra space.

## Not adaptive

The scan always examines the whole suffix, so sorted input costs exactly the same  $n(n - 1) / 2$  comparisons as random input.

## Fewest swaps

At most one swap per iteration, so at most  $n - 1$  swaps in total, the smallest of the simple sorts. Bubble sort can need up to  $n(n - 1) / 2$ .

## Comparison-based

Every decision comes from comparing two values.

*Sorting / Selection sort · Why those complexities occur*

# Why those complexities occur

## Every case · $O(n^2)$ comparisons

The scan for iteration  $i$  always looks at  $n - 1 - i$  cells, whatever the values are.

## Swaps · at most $n - 1$

One per iteration, so the writing is only  $O(n)$ : that is selection sort's one real advantage.

## The comparison staircase never changes

|         |                                                                                   |   |
|---------|-----------------------------------------------------------------------------------|---|
| $i = 1$ |  | 4 |
| $i = 2$ |  | 3 |
| $i = 3$ |  | 2 |
| $i = 4$ |  | 1 |

total =  $4 + 3 + 2 + 1 = 10$

The staircase is exactly the same one bubble sort walks in its worst case, and for the same reason it totals  $n(n - 1) / 2$ . The difference is that for selection sort this is the cost *every time*: there is no best case that avoids it.

$$(n - 1) + (n - 2) + \dots + 1 = n(n - 1) / 2 \rightarrow O(n^2)$$

comparisons, always

| N (VALUES) | COMPARISONS | MAXIMUM SWAPS |
|------------|-------------|---------------|
| 5          | 10          | 4             |
| 10         | 45          | 9             |
| 100        | 4,950       | 99            |
| 1,000      | 499,500     | 999           |

Read across the row for 1,000 values: nearly half a million comparisons but never more than 999 swaps. If a swap were expensive (large records, slow storage) that gap would matter; if comparisons are the cost, selection sort has no way to save any.

## Space

Only  $i$ ,  $j$ ,  $\text{minIndex}$  and  $\text{temp}$ :  $O(1)$ .

## When to use it

### A good choice

Small arrays; situations where

#### writing

is much more expensive than reading (flash memory, very large records, writing to a file); when you need a predictable, data-independent running time; and when simplicity matters more than speed.

### A poor choice

Large arrays ( $O(n^2)$  comparisons); data that is already or nearly sorted (it cannot take advantage); anywhere stability is needed, for example sorting records by one field while keeping an earlier order for ties.

Compared with its relatives: **insertion sort** beats it on nearly sorted data and is stable; **bubble sort** makes many more swaps; **merge sort** is far faster on large arrays. Selection sort is a good answer to “which simple sort minimises swaps?”, and almost nothing else.

## Sorting · Why sort data at all?

## Why sort data at all?

Before the algorithms: what does sorted data give you that unsorted data does not, and what does it cost?

Everything in this chapter is about putting values in order. Before learning *how*, it is worth being clear about **why**, because it is a favourite exam question and because it explains why searching and sorting are taught together.

**Short answer for a test:** sorting data makes it easier and faster to find information in it. A sorted array can be searched with binary search, in  $O(\log n)$  instead of  $O(n)$ .

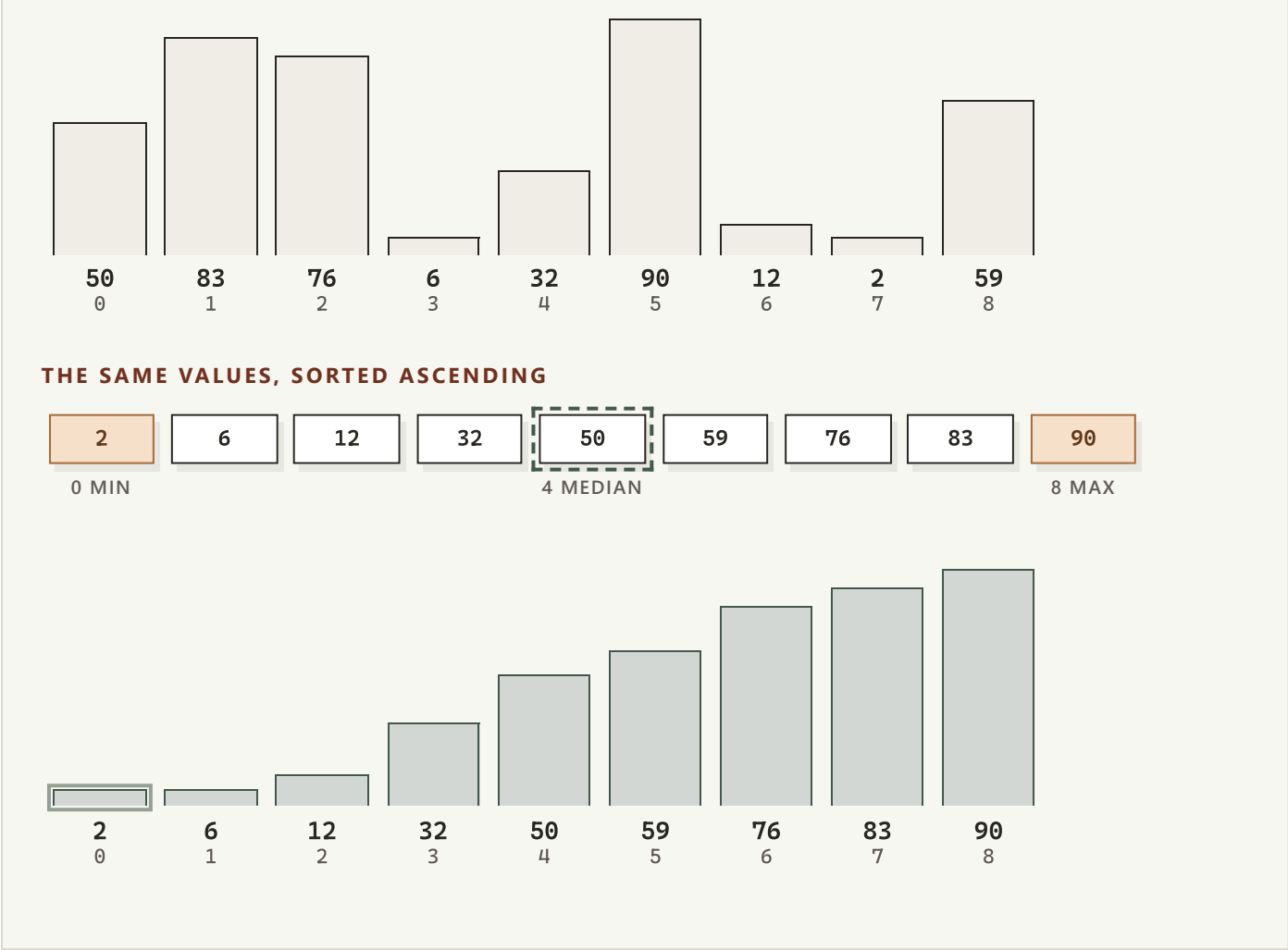
## What becomes easy once data is sorted

Take the nine values from the past paper. Unsorted they tell you nothing at a glance; sorted, a lot of questions answer themselves.

### UNSORTED

|    |    |    |   |    |    |    |   |    |
|----|----|----|---|----|----|----|---|----|
| 50 | 83 | 76 | 6 | 32 | 90 | 12 | 2 | 59 |
| 0  | 1  | 2  | 3 | 4  | 5  | 6  | 7 | 8  |





| QUESTION                    | UNSORTED ARRAY                     | SORTED ARRAY                                            |
|-----------------------------|------------------------------------|---------------------------------------------------------|
| Smallest and largest value? | Look at all 9 cells                | The first and last cell: 2 and 90                       |
| Median (middle) value?      | Needs a lot of work                | The middle cell: 50                                     |
| Is a value present?         | Linear search: up to 9 comparisons | Binary search: at most 4 probes                         |
| Where are the duplicates?   | Anywhere                           | Next to each other (see below)                          |
| Merge two lists?            | No shortcut                        | Merge them by zipping (that is merge sort's merge step) |
| Show a list to a person?    | Hard to read                       | Alphabetical or numerical order, easy to scan           |

## Duplicates end up side by side

Finding repeated values in [3, 1, 3, 2, 3] means comparing every pair. After sorting, [1, 2, 3, 3, 3], equal values are neighbours, so one pass comparing each cell with the next finds them all.

# Sorting is not free: is it worth it?

Sorting costs about  $n \log_2 n$  steps, and it only pays off if you search more than a few times. Suppose you have **1,000,000** values and will search for **1,000** different keys:

| STRATEGY                                 | WORK                                                                    | TOTAL                |
|------------------------------------------|-------------------------------------------------------------------------|----------------------|
| Never sort: linear search every time     | 1,000 searches $\times$ up to 1,000,000 comparisons                     | <b>1,000,000,000</b> |
| Sort once, then binary search every time | sort $\approx$ 20,000,000 steps, then 1,000 searches $\times$ 20 probes | <b>20,020,000</b>    |

That is about **49 $\times$  less work** after sorting. With a single search the answer flips: scanning once costs 1,000,000 and sorting first costs 20,000,000. The rule: *search once, scan; search often, sort first*.

▼ Quick check: Exam-style question 3(a), “What is the purpose of sorting data?”

To arrange data in a defined order (ascending or descending) so that information is easier and faster to find, for example by using binary search instead of scanning every element. Other valid points: duplicates become adjacent, minimum and maximum are at the ends, and sorted data is easier to read and merge.

## Sorting

# The sorting map

Every algorithm orders values, but each uses a different repeated action. The four are taught in this order: Bubble, Selection, Insertion, Merge.

| ALGORITHM | REPEATED ACTION                                  | TYPICAL TIME  | BEST FIT                         |
|-----------|--------------------------------------------------|---------------|----------------------------------|
| Bubble    | Compare neighbours; swap wrong pairs             | $O(n^2)$      | Learning and tiny inputs         |
| Selection | Find the smallest remaining value; place it next | $O(n^2)$      | When few swaps matter            |
| Insertion | Shift larger sorted values; insert a key         | $O(n^2)$      | Small or nearly sorted data      |
| Merge     | Divide into halves; merge sorted halves          | $O(n \log n)$ | Large data and predictable speed |

# Vocabulary behind every property tile

|                                                                                                                              |                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Comparison-based</b><br>Order is decided only by asking questions such as $a < b$ .                                       | <b>Best / average / worst</b><br>The same algorithm on three arrangements of the same $n$ values: already ordered, typical, and the most awkward. |
| <b>In-place</b><br>Values are rearranged inside the original array using only a few extra variables: $O(1)$ auxiliary space. | <b>Auxiliary space</b><br>Extra memory beyond the input array, such as the temporary copies merge sort makes.                                     |
| <b>Adaptive</b><br>Does less work when the input is already or nearly sorted.                                                | <b>Stable</b><br>Equal values keep their original relative order.                                                                                 |

## Why stability matters

Sort these students by mark. Ana and Ben both scored 70, and Ana was listed first.

Start

|     |     |    |
|-----|-----|----|
| 70  | 70  | 65 |
| ANA | BEN | CY |

Stable sort result

|    |     |     |
|----|-----|-----|
| 65 | 70  | 70  |
| CY | ANA | BEN |

An unstable sort may give

|    |     |     |
|----|-----|-----|
| 65 | 70  | 70  |
| CY | BEN | ANA |

Both outputs are sorted by mark. Only the stable one keeps Ana before Ben, which matters when you sort by one field and want an earlier ordering by another field preserved.

## Count the disorder: inversions

An **inversion** is a pair of positions whose values are in the wrong order: a larger value sits to the left of a smaller one.  $[4, 2, 5, 1, 3]$  has six:  $(4,2)$ ,  $(4,1)$ ,  $(4,3)$ ,  $(2,1)$ ,  $(5,1)$ ,  $(5,3)$ .

| ALGORITHM | WHAT IS COUNTED | ON $[4, 2, 5, 1, 3]$ | LINK TO INVERSIONS                                                     |
|-----------|-----------------|----------------------|------------------------------------------------------------------------|
| Bubble    | adjacent swaps  | 6                    | Each swap of neighbours removes exactly one inversion.                 |
| Insertion | shifts          | 6                    | Each shift removes exactly one inversion.                              |
| Selection | swaps           | 2                    | At most $n - 1$ swaps; one swap can remove several inversions at once. |

A fully reversed array of  $n$  values has  $n(n - 1) / 2$  inversions. For  $[5, 4, 3, 2, 1]$  that is 10, which equals the 10 shifts in the reverse-order insertion example. This is the reason bubble and insertion sort are  $O(n^2)$  in the worst case.

Sorting / Selection sort · Descending order

# Descending order

To sort largest to smallest, select the **largest** value of the suffix instead of the smallest. Only the comparison changes, and the variable is best renamed so the code still reads honestly.

Ascending: pick the minimum

```
if (arr[j] < arr[minIndex]) {
  minIndex = j;
}
```

Descending: pick the maximum

```
if (arr[j] > arr[maxIndex]) {
  maxIndex = j;
}
```

| ITERATION | ASCENDING (CHOOSES THE SMALLEST) | DESCENDING (CHOOSES THE LARGEST) |
|-----------|----------------------------------|----------------------------------|
| i = 0     | [1, 2, 5, 4, 3] · swap           | [5, 2, 4, 1, 3] · swap           |
| i = 1     | [1, 2, 5, 4, 3] · no swap        | [5, 4, 2, 1, 3] · swap           |
| i = 2     | [1, 2, 3, 4, 5] · swap           | [5, 4, 3, 1, 2] · swap           |
| i = 3     | [1, 2, 3, 4, 5] · no swap        | [5, 4, 3, 2, 1] · swap           |

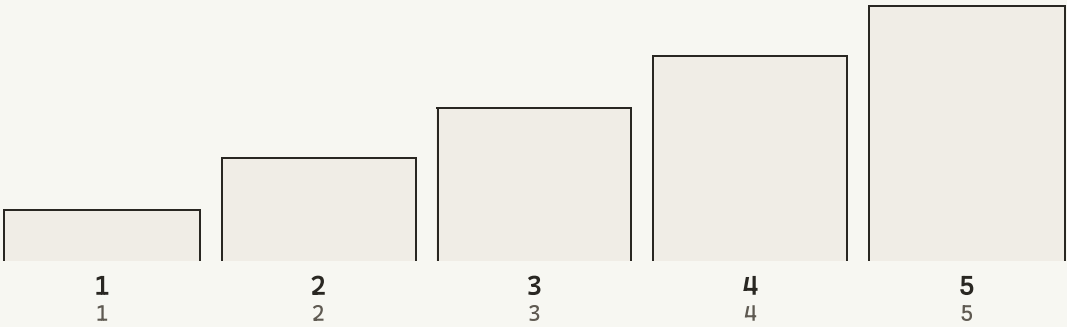
Ascending made **2 swaps**, descending **4**, and both made the same **10 comparisons**. The comparison count never depends on direction or on the data, only the number of swaps does.

Sorting / Selection sort · Worked examples

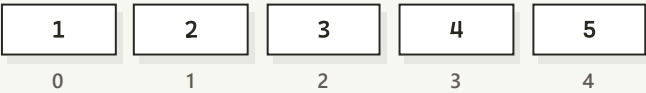
# Worked examples

Five arrays: no disorder, a reversed array, the most swaps, duplicates and a longer run.

Already sorted · [1, 2, 3, 4, 5]  
Every minimum is already sitting at the boundary.



Start: nothing is sorted



After  $i = 0$  · cells 0..0 are final



After  $i = 1$  · cells 0..1 are final



After  $i = 2$  · cells 0..2 are final



After  $i = 3$  · cells 0..3 are final



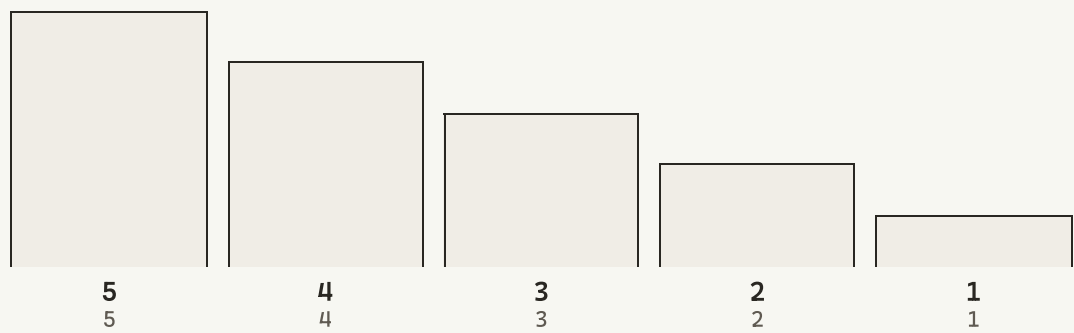
| I     | CHOICE | ACTION | ARRAY AFTER THE ITERATION | COMPARISONS |
|-------|--------|--------|---------------------------|-------------|
| Start | —      | —      | [1, 2, 3, 4, 5]           | —           |

| I     | CHOICE                                        | ACTION                    | ARRAY AFTER THE ITERATION | COMPARISONS |
|-------|-----------------------------------------------|---------------------------|---------------------------|-------------|
| 0     | smallest of the unsorted part is 1 at index 0 | already in place: no swap | [1, 2, 3, 4, 5]           | 4           |
| 1     | smallest of the unsorted part is 2 at index 1 | already in place: no swap | [1, 2, 3, 4, 5]           | 3           |
| 2     | smallest of the unsorted part is 3 at index 2 | already in place: no swap | [1, 2, 3, 4, 5]           | 2           |
| 3     | smallest of the unsorted part is 4 at index 3 | already in place: no swap | [1, 2, 3, 4, 5]           | 1           |
| Total |                                               | 0 swaps                   | sorted                    | 10          |

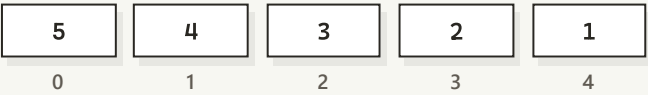
**10 comparisons and 0 swaps. The data is perfectly sorted and selection sort still makes all 10 comparisons: it cannot know the answer without looking. That is why it is not adaptive.**

Reverse order · [5, 4, 3, 2, 1]

A surprise: the worst-looking input is not the worst case for swaps.



Start: nothing is sorted



After  $i = 0$  · cells 0..0 are final



After  $i = 1$  · cells 0..1 are final



After  $i = 2$  · cells 0..2 are final



After  $i = 3$  · cells 0..3 are final

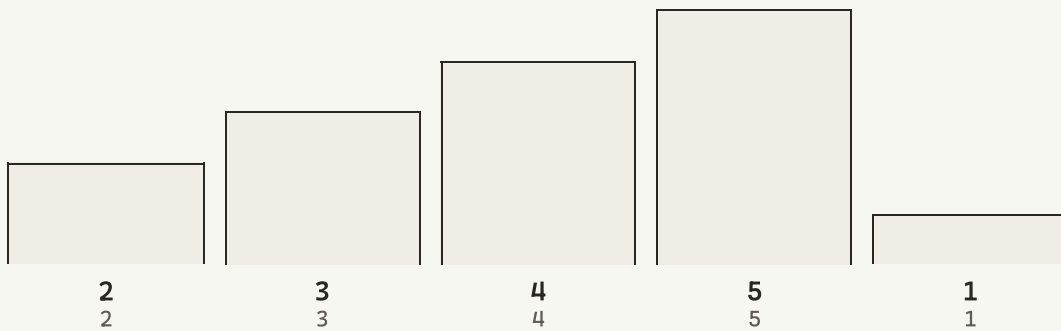


| I     | CHOICE                                        | ACTION                    | ARRAY AFTER THE ITERATION | COMPARISONS |
|-------|-----------------------------------------------|---------------------------|---------------------------|-------------|
| Start | —                                             | —                         | [5, 4, 3, 2, 1]           | —           |
| 0     | smallest of the unsorted part is 1 at index 4 | swap index 0 with index 4 | [1, 4, 3, 2, 5]           | 4           |
| 1     | smallest of the unsorted part is 2 at index 3 | swap index 1 with index 3 | [1, 2, 3, 4, 5]           | 3           |
| 2     | smallest of the unsorted part is 3 at index 2 | already in place: no swap | [1, 2, 3, 4, 5]           | 2           |
| 3     | smallest of the unsorted part is 4 at index 3 | already in place: no swap | [1, 2, 3, 4, 5]           | 1           |
| Total |                                               | 2 swaps                   | sorted                    | 10          |

10 comparisons but only 2 swaps. Each swap sends one value to each end at the same time, and the middle value needs nothing, so a reversed array needs about  $n / 2$  swaps.

The most swaps · [2, 3, 4, 5, 1]

The minimum is at the far end, then the next minimum is at the far end of what remains, and so on.



Start: nothing is sorted



After  $i = 0$  · cells 0..0 are final



After  $i = 1$  · cells 0..1 are final



After  $i = 2$  · cells 0..2 are final





After  $i = 3$  · cells 0..3 are final

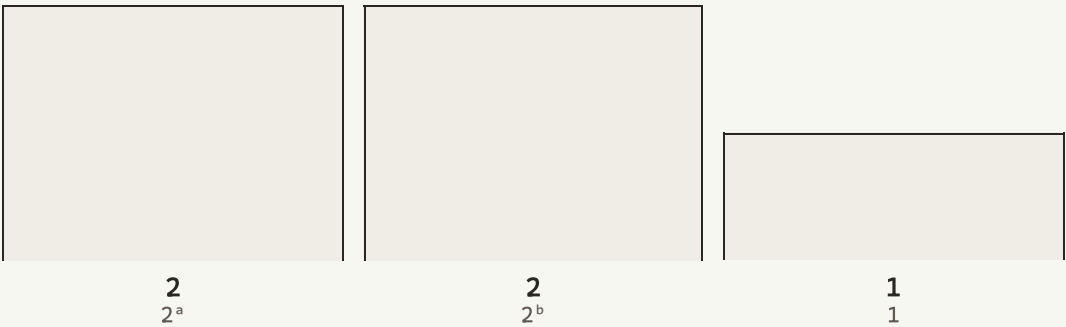


| I     | CHOICE                                        | ACTION                    | ARRAY AFTER THE ITERATION | COMPARISONS |
|-------|-----------------------------------------------|---------------------------|---------------------------|-------------|
| Start | —                                             | —                         | [2, 3, 4, 5, 1]           | —           |
| 0     | smallest of the unsorted part is 1 at index 4 | swap index 0 with index 4 | [1, 3, 4, 5, 2]           | 4           |
| 1     | smallest of the unsorted part is 2 at index 4 | swap index 1 with index 4 | [1, 2, 4, 5, 3]           | 3           |
| 2     | smallest of the unsorted part is 3 at index 4 | swap index 2 with index 4 | [1, 2, 3, 5, 4]           | 2           |
| 3     | smallest of the unsorted part is 4 at index 4 | swap index 3 with index 4 | [1, 2, 3, 4, 5]           | 1           |
| Total |                                               | 4 swaps                   | sorted                    | 10          |

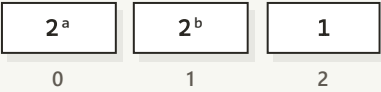
10 comparisons and 4 swaps, which is the maximum  $n - 1$ . Even this worst case for swaps is tiny next to the 10 comparisons.

### Duplicates · $[2^a, 2^b, 1]$

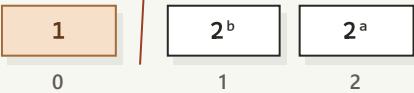
Two equal values, labelled so we can follow them.



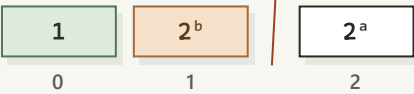
Start: nothing is sorted



After  $i = 0$  · cells 0..0 are final



After  $i = 1$  · cells 0..1 are final

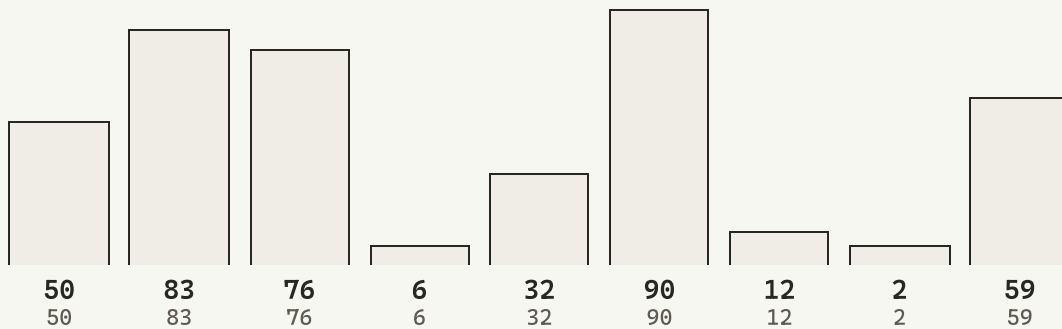


| I     | CHOICE                                            | ACTION                    | ARRAY AFTER THE ITERATION | COMPARISONS |
|-------|---------------------------------------------------|---------------------------|---------------------------|-------------|
| Start | —                                                 | —                         | $[2^a, 2^b, 1]$           | —           |
| 0     | smallest of the unsorted part is 1 at index 2     | swap index 0 with index 2 | $[1, 2^b, 2^a]$           | 2           |
| 1     | smallest of the unsorted part is $2^b$ at index 1 | already in place: no swap | $[1, 2^b, 2^a]$           | 1           |
| Total |                                                   | 1 swaps                   | sorted                    | 3           |

The result is  $[1, 2^b, 2^a]$ : the two 2s came out in the *opposite* order. The first swap threw  $2^a$  to the far end, past  $2^b$ . So selection sort is **not stable**.

A longer array ·  $[50, 83, 76, 6, 32, 90, 12, 2, 59]$  (nine values)

The array from the past paper, question 3. The table shows every iteration; try to predict the minimum before reading each row.



| I     | CHOICE                                         | ACTION                    | ARRAY AFTER THE ITERATION          | COMPARISONS |
|-------|------------------------------------------------|---------------------------|------------------------------------|-------------|
| Start | —                                              | —                         | [50, 83, 76, 6, 32, 90, 12, 2, 59] | —           |
| 0     | smallest of the unsorted part is 2 at index 7  | swap index 0 with index 7 | [2, 83, 76, 6, 32, 90, 12, 50, 59] | 8           |
| 1     | smallest of the unsorted part is 6 at index 3  | swap index 1 with index 3 | [2, 6, 76, 83, 32, 90, 12, 50, 59] | 7           |
| 2     | smallest of the unsorted part is 12 at index 6 | swap index 2 with index 6 | [2, 6, 12, 83, 32, 90, 76, 50, 59] | 6           |
| 3     | smallest of the unsorted part is 32 at index 4 | swap index 3 with index 4 | [2, 6, 12, 32, 83, 90, 76, 50, 59] | 5           |
| 4     | smallest of the unsorted part is 50 at index 7 | swap index 4 with index 7 | [2, 6, 12, 32, 50, 90, 76, 83, 59] | 4           |
| 5     | smallest of the unsorted part is 59 at index 8 | swap index 5 with index 8 | [2, 6, 12, 32, 50, 59, 76, 83, 90] | 3           |
| 6     | smallest of the unsorted part is 76 at index 6 | already in place: no swap | [2, 6, 12, 32, 50, 59, 76, 83, 90] | 2           |
| 7     | smallest of the unsorted part is 83 at index 7 | already in place: no swap | [2, 6, 12, 32, 50, 59, 76, 83, 90] | 1           |
| Total |                                                | 6 swaps                   | sorted                             | 36          |

**36 comparisons (always  $n(n - 1) / 2$ ) and 6 swaps (at most  $n - 1 = 8$ ).**

# Common mistakes

---

- 1 Storing the minimum value instead of its index.**  
To swap you need the position. Keep `minIndex`.

---

- 2 Starting the scan at `i` instead of `i + 1`.**  
It still works, because a value compared with itself is never smaller, but it wastes one comparison per iteration and changes your counts.

---

- 3 Swapping inside the scan.**  
Swap *after* the scan finishes, once you know the true minimum. Swapping every time you find a smaller value turns it into a different (and slower) algorithm.

---

- 4 Forgetting that the minimum may already be at `i`.**  
Then `minIndex == i` and no swap is needed. Counting a swap here gives a wrong total.

---

- 5 Running the outer loop to `n` instead of `n - 1`.**  
When only one cell is left it is automatically correct, so the last iteration is wasted.

---

- 6 Expecting it to stop early on sorted data.**  
It cannot. All  $n(n - 1) / 2$  comparisons are made whatever the input looks like.

---

- 7 Calling it stable.**  
It is not: the long-distance swap can reorder equal values.

---

- 8 Using the wrong comparison for descending order.**  
Select the *maximum* (`>`); do not run the ascending sort and reverse the answer unless the question asks for it.

*Sorting / Selection sort · A test-ready summary*

# A test-ready summary

## TEST ANSWER

Selection sort is an in-place, comparison-based sorting algorithm that is not stable and not adaptive. It splits the array into a sorted prefix and an unsorted suffix. In each iteration it scans the whole suffix to find the smallest value, remembers its index, and swaps it into the first position of the suffix, so the prefix grows by one final cell. It makes exactly  $n(n - 1) / 2$  comparisons whatever the input, but at most  $n - 1$  swaps. Its best, average and worst cases are all  $O(n^2)$  with  $O(1)$  extra space. It is useful for small arrays and when swaps are expensive, but not for large data.

## Sorting / Selection sort · Practice

### Practice

#### Guided question

Use selection sort (ascending) on [6, 3, 8, 4]. For each iteration say which value is selected, where it was, and whether a swap happened.

#### ▼ Reveal the guided answer

**i = 0:** scan indexes 1..3; smallest is 3 at index 1 → swap → [3, 6, 8, 4]

**i = 1:** scan indexes 2..3; smallest is 4 at index 3 → swap → [3, 4, 8, 6]

**i = 2:** scan indexes 3..3; smallest is 6 at index 3 → swap → [3, 4, 6, 8]

**Totals:** 6 comparisons and 3 swaps.

#### Independent question

Trace selection sort on [9, 4, 7, 3, 5], writing the array after every iteration and the totals.

- ☐ For each **i**, find the smallest value in the unsorted part and its index.
- ☐ Say whether a swap is needed.

- Write the array after the iteration.
- Total the comparisons (it should be 10 without tracing them) and the swaps.

▼ **Reveal the independent answer**

| I            | CHOICE                                        | ACTION                    | ARRAY AFTER THE ITERATION | COMPARISONS |
|--------------|-----------------------------------------------|---------------------------|---------------------------|-------------|
| Start        | —                                             | —                         | [9, 4, 7, 3, 5]           | —           |
| 0            | smallest of the unsorted part is 3 at index 3 | swap index 0 with index 3 | [3, 4, 7, 9, 5]           | 4           |
| 1            | smallest of the unsorted part is 4 at index 1 | already in place: no swap | [3, 4, 7, 9, 5]           | 3           |
| 2            | smallest of the unsorted part is 5 at index 4 | swap index 2 with index 4 | [3, 4, 5, 9, 7]           | 2           |
| 3            | smallest of the unsorted part is 7 at index 4 | swap index 3 with index 4 | [3, 4, 5, 7, 9]           | 1           |
| <b>Total</b> |                                               | <b>3 swaps</b>            | <b>sorted</b>             | <b>10</b>   |

## Test-style question

Show the contents of [50, 83, 76, 6, 32, 90, 12, 2, 59] after each iteration when sorting **descending** with selection sort.

▼ **Reveal the test-style answer**

| I     | CHOICE                                        | ACTION                    | ARRAY AFTER THE ITERATION          | COMPARISONS |
|-------|-----------------------------------------------|---------------------------|------------------------------------|-------------|
| Start | —                                             | —                         | [50, 83, 76, 6, 32, 90, 12, 2, 59] | —           |
| 0     | largest of the unsorted part is 90 at index 5 | swap index 0 with index 5 | [90, 83, 76, 6, 32, 50, 12, 2, 59] | 8           |
| 1     | largest of the unsorted part is 83 at index 1 | already in place: no swap | [90, 83, 76, 6, 32, 50, 12, 2, 59] | 7           |

| I     | CHOICE                                        | ACTION                    | ARRAY AFTER THE ITERATION          | COMPARISONS |
|-------|-----------------------------------------------|---------------------------|------------------------------------|-------------|
| 2     | largest of the unsorted part is 76 at index 2 | already in place: no swap | [90, 83, 76, 6, 32, 50, 12, 2, 59] | 6           |
| 3     | largest of the unsorted part is 59 at index 8 | swap index 3 with index 8 | [90, 83, 76, 59, 32, 50, 12, 2, 6] | 5           |
| 4     | largest of the unsorted part is 50 at index 5 | swap index 4 with index 5 | [90, 83, 76, 59, 50, 32, 12, 2, 6] | 4           |
| 5     | largest of the unsorted part is 32 at index 5 | already in place: no swap | [90, 83, 76, 59, 50, 32, 12, 2, 6] | 3           |
| 6     | largest of the unsorted part is 12 at index 6 | already in place: no swap | [90, 83, 76, 59, 50, 32, 12, 2, 6] | 2           |
| 7     | largest of the unsorted part is 6 at index 8  | swap index 7 with index 8 | [90, 83, 76, 59, 50, 32, 12, 6, 2] | 1           |
| Total |                                               | 4 swaps                   | sorted                             | 36          |

## More worked cases: edge cases, duplicates and descending order

Each case has input and result diagrams, evaluated decisions or merge inputs, exact counts, and a final state.

1. Nearly sorted

1

2

4

3

5

01234

↓ ascending selection sort

1

2

3

4

5

01234

The trace below connects the input diagram to the final result.

| PASS | BEFORE      | EVALUATED CONDITIONS                                                                                                                                         | AFTER       | KEY COMPARISONS |
|------|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-----------------|
| 1    | [1,2,4,3,5] | 2 < 1 → false; best stays 0; 4 < 1 → false; best stays 0; 3 < 1 → false; best stays 0; 5 < 1 → false; best stays 0; Minimum is already in position; no swap. | [1,2,4,3,5] | 4               |
| 2    | [1,2,4,3,5] | 4 < 2 → false; best stays 1; 3 < 2 → false; best stays 1; 5 < 2 → false; best stays 1; Minimum is already in position; no swap.                              | [1,2,4,3,5] | 3               |
| 3    | [1,2,4,3,5] | 3 < 4 → true; best index becomes 3; 5 < 3 → false; best stays 3; Scan is complete: swap indexes 2 and 3.                                                     | [1,2,3,4,5] | 2               |
| 4    | [1,2,3,4,5] | 5 < 4 → false; best stays 3; Minimum is already in position; no swap.                                                                                        | [1,2,3,4,5] | 1               |

10 key comparisons; 1 swaps.



2. Reverse order

5

4

3

2

1

01234

↓ ascending selection sort

1

2

3

4

5

01234

The trace below connects the input diagram to the final result.

| PASS | BEFORE      | EVALUATED CONDITIONS                                                                                                                                                                    | AFTER       | KEY COMPARISONS |
|------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|-----------------|
| 1    | [5,4,3,2,1] | 4 < 5 → true; best index becomes 1; 3 < 4 → true; best index becomes 2; 2 < 3 → true; best index becomes 3; 1 < 2 → true; best index becomes 4; Scan is complete: swap indexes 0 and 4. | [1,4,3,2,5] | 4               |
| 2    | [1,4,3,2,5] | 3 < 4 → true; best index becomes 2; 2 < 3 → true; best index becomes 3; 5 < 2 → false; best stays 3; Scan is complete: swap indexes 1 and 3.                                            | [1,2,3,4,5] | 3               |
| 3    | [1,2,3,4,5] | 4 < 3 → false; best stays 2; 5 < 3 → false; best stays 2; Minimum is already in position; no swap.                                                                                      | [1,2,3,4,5] | 2               |
| 4    | [1,2,3,4,5] | 5 < 4 → false; best stays 3; Minimum is already in position; no swap.                                                                                                                   | [1,2,3,4,5] | 1               |

10 key comparisons; 2 swaps.

3. Duplicate values

2

2

1

3

0123

↓ ascending selection sort

1

2

2

3

0123

The trace below connects the input diagram to the final result.

| PASS | BEFORE    | EVALUATED CONDITIONS                                                                                                                  | AFTER     | KEY COMPARISONS |
|------|-----------|---------------------------------------------------------------------------------------------------------------------------------------|-----------|-----------------|
| 1    | [2,2,1,3] | 2 < 2 → false; best stays 0; 1 < 2 → true; best index becomes 2; 3 < 1 → false; best stays 2; Scan is complete: swap indexes 0 and 2. | [1,2,2,3] | 3               |
| 2    | [1,2,2,3] | 2 < 2 → false; best stays 1; 3 < 2 → false; best stays 1; Minimum is already in position; no swap.                                    | [1,2,2,3] | 2               |
| 3    | [1,2,2,3] | 3 < 2 → false; best stays 2; Minimum is already in position; no swap.                                                                 | [1,2,2,3] | 1               |

6 key comparisons; 1 swaps.

4. Uneven length

38

27

43

3

9

01234

↓ ascending selection sort

3

9

27

38

43

01234

The trace below connects the input diagram to the final result.

| PASS | BEFORE         | EVALUATED CONDITIONS                                                                                                                                                           | AFTER          | KEY COMPARISONS |
|------|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|-----------------|
| 1    | [38,27,43,3,9] | 27 < 38 → true; best index becomes 1; 43 < 27 → false; best stays 1; 3 < 27 → true; best index becomes 3; 9 < 3 → false; best stays 3; Scan is complete: swap indexes 0 and 3. | [3,27,43,38,9] | 4               |
| 2    | [3,27,43,38,9] | 43 < 27 → false; best stays 1; 38 < 27 → false; best stays 1; 9 < 27 → true; best index becomes 4; Scan is complete: swap indexes 1 and 4.                                     | [3,9,43,38,27] | 3               |
| 3    | [3,9,43,38,27] | 38 < 43 → true; best index becomes 3; 27 < 38 → true; best index becomes 4; Scan is complete: swap indexes 2 and 4.                                                            | [3,9,27,38,43] | 2               |
| 4    | [3,9,27,38,43] | 43 < 38 → false; best stays 3; Minimum is already in position; no swap.                                                                                                        | [3,9,27,38,43] | 1               |

10 key comparisons; 3 swaps.

5. Descending target

8

3

5

1

0123

↓ descending selection sort

8

5

3

1

0123

The trace below connects the input diagram to the final result.

| PASS | BEFORE    | EVALUATED CONDITIONS                                                                                                            | AFTER     | KEY COMPARISONS |
|------|-----------|---------------------------------------------------------------------------------------------------------------------------------|-----------|-----------------|
| 1    | [8,3,5,1] | 3 > 8 → false; best stays 0; 5 > 8 → false; best stays 0; 1 > 8 → false; best stays 0; Minimum is already in position; no swap. | [8,3,5,1] | 3               |
| 2    | [8,3,5,1] | 5 > 3 → true; best index becomes 2; 1 > 5 → false; best stays 2; Scan is complete: swap indexes 1 and 2.                        | [8,5,3,1] | 2               |
| 3    | [8,5,3,1] | 1 > 3 → false; best stays 2; Minimum is already in position; no swap.                                                           | [8,5,3,1] | 1               |

6 key comparisons; 1 swaps. Select the maximum instead: change `arr[j] < arr[min]` to `arr[j] > arr[min]`. The prefix then grows in descending order.

## Exam-style practice with worked answers

### ▼ A. Trace [8, 3, 5, 1] in descending order. Show every completed pass/merge. [8]

| PASS | BEFORE    | AFTER     | KEY COMPARISONS |
|------|-----------|-----------|-----------------|
| 1    | [8,3,5,1] | [8,3,5,1] | 3               |
| 2    | [8,3,5,1] | [8,5,3,1] | 2               |
| 3    | [8,5,3,1] | [8,5,3,1] | 1               |

Final result [8,5,3,1]. Select the maximum instead: change  $\text{arr}[j] < \text{arr}[\text{min}]$  to  $\text{arr}[j] > \text{arr}[\text{min}]$ . The prefix then grows in descending order.

### ▼ B. Explain stability with tagged equal keys. [4]

Swap 1 with 2a in [2a, 2b, 1]: result [1, 2b, 2a]. The equal records crossed, so the taught version is not stable.

### ▼ C. Give the time and space, then justify both. [5]

Best / average / worst:  $O(n^2)$ . Every suffix is fully scanned, even on sorted input. Key comparisons:  $n(n-1)/2$ . Lengths  $n-1, n-2, \dots, 1$ . Swaps: At most  $n-1$ . At most one actual exchange per pass; self-swaps are skipped. Auxiliary space:  $O(1)$  iterative;  $O(n)$  recursive. Only index variables and a temporary value in the iterative form. Stable / in place / adaptive: No / yes / no. A long swap can cross equal records; scanning does not shrink with presortedness.

▼ D. Debug the code before tracing it. [4]

| MISTAKE                            | SYMPTOM                                                 | FIX                                            |
|------------------------------------|---------------------------------------------------------|------------------------------------------------|
| Set min = arr[i]                   | Treats a value as an index.                             | Set min = i, then compare arr[j] and arr[min]. |
| Swap every new minimum immediately | Changes the taught selection procedure and swap counts. | Finish the full scan, then swap once.          |

▼ E. What happens for [] and [7]? [2]

Both are already sorted. No key comparisons or value movements are required. Merge sort returns at its base case; the iterative methods never enter a pass.

---

## Topic quiz: test your understanding

Answer all 12 questions. Choose an option to see the reasoning; reset to retry. The printable answer key follows the questions.

1. What is the last valid index of an array of length 6?

- ☐ A. 5
- ☐ B. 6
- ☐ C. 7

2. What does  $O(1)$  auxiliary space describe?

- ☐ A. Exactly one machine instruction
- ☐ B. The input contains one value
- ☐ C. Extra storage stays bounded as  $n$  grows

**3. What is a loop invariant?**

- ☐ A. A loop that never terminates
- ☐ B. A statement that remains true at a specified loop boundary
- ☐ C. A variable that never changes

**4. What does stable sorting preserve?**

- ☐ A. The original relative order of records with equal keys
- ☐ B. Every original array index
- ☐ C. The original order of unequal keys

**5. What should a correct ascending sort do with [] and [7]?**

- ☐ A. Throw an error for both
- ☐ B. Add a zero to each
- ☐ C. Leave both unchanged

**6. When does ordinary selection sort swap?**

- ☐ A. Before looking at the suffix
- ☐ B. After the complete suffix scan
- ☐ C. Every time a new minimum is seen

**7. What does min store?**

- ☐ A. The index of the best candidate
- ☐ B. The minimum value itself
- ☐ C. The number of swaps

8. Opening map: the first selected minimum and destination?

- ☐ A. 29 goes to index 8
- ☐ B. 36 goes to index 0
- ☐ C. 13 goes to index 0

9. What happens on the opening map's sixth pass?

- ☐ A. The algorithm stops scanning permanently
- ☐ B. 66 is already in position; no swap
- ☐ C. 66 swaps with 98

10. Comparisons on a sorted five-value input?

- ☐ A. 10
- ☐ B. 4
- ☐ C. 0

11. Maximum actual swaps for  $n$  values when self-swaps are skipped?

- ☐ A.  $n(n - 1)/2$
- ☐ B. Always zero
- ☐ C.  $n - 1$

12. What does sorting [2a, 2b, 1] demonstrate?

- ☐ A. Duplicates prevent termination
- ☐ B. Long swaps can reverse equal-key order
- ☐ C. Selection sort is stable



### ▼ Answer key and explanations

1. **5.** Indexes start at 0, so the last index is  $\text{length} - 1$ .
2. **Extra storage stays bounded as  $n$  grows.** Auxiliary space counts extra working storage, excluding the input array.
3. **A statement that remains true at a specified loop boundary.** The invariant explains why each iteration preserves progress toward a correct result.
4. **The original relative order of records with equal keys.** Tags such as 2a and 2b let you check whether equal-key records crossed.
5. **Leave both unchanged.** Both inputs are already sorted; no key comparison is necessary.
6. **After the complete suffix scan.** The scan remembers an index; the final minimum is moved only after scanning.
7. **The index of the best candidate.** `arr[min]` accesses the candidate value, whereas `min` identifies its position.
8. **13 goes to index 0.** 13 is the smallest of all nine values. It swaps with 29.
9. **66 is already in position; no swap.** A no-swap pass fixes one position; it does not prove the entire remaining suffix sorted.
10. **10.** Selection sort still scans suffixes of lengths 4, 3, 2, 1.
11.  **$n - 1$ .** There is at most one exchange per outer pass.
12. **Long swaps can reverse equal-key order.** The first swap yields [1, 2b, 2a], so this ordinary version is unstable.